

Big Data Project 2 presentation Script

Jakob Sverre Alexandersen *
FSN Labs
ja.al@fsncapital.com

November 5, 2025



*This script is so incredibly baller omd

Contents

1	Introduction	3
1.1	CNN Justification	3
2	CNN theory	3
2.1	Convolutional Operation	3
2.2	Architecture Breakdown	3
2.2.1	Part 1: Convolutional Layers	3
2.2.2	Part 2: Fully Connected Layers	4
2.3	Briefly show the network	4
2.4	Training	4
2.4.1	Gradient Descent	4
2.4.2	Epochs	4
2.5	Evaluation	4
2.6	Side note on overfitting: double descent	5
3	Implementation	5
3.1	Visualization	5
4	K-fold Cross-Validation	5
5	Results	6

1 Introduction

Today I'm going to present a CNN, otherwise known as a convolutional neural network.

First I'll give you guys an overview of the problem I'm trying to solve, and then walk you through how I solved it.

So the problem I am solving here is image recognition, mainly digit classifying based on images. The dataset I am using is the MNIST dataset which I'm sure you've seen before, and we'll take a look at it later.

Before I walk you through my CNN image classifier architecture, I want to justify why I'm using a CNN and not some other architecture. In addition to that, it's important to thoroughly understand how they work on a basic level, so I'll give you a brief overview on how CNNs actually work.

1.1 CNN Justification

The main reason why I chose a CNN for this task is mainly because they are generally known to just work great for images.

If we go in depth as to why they're actually known for being so good, it's mainly because of their local receptive fields, which I will talk a little about later. In addition to that, they employ weight sharing, which allows the model to detect the same feature, like a stroke or corner, anywhere in the image and not just in one position. In turn, this leads to the model having far less parameters than other networks, like Fully Connected Networks.

I also mention hierarchical feature learning here. That only means that it learns low-level features like blobs and edges in early layers and then later on it learns high-level parts, like shapes and parts of digits in this case.

2 CNN theory

2.1 Convolutional Operation

So how does it all work? This formula represents the convolution operation in CNNs. All I want you to pay attention to now is this K .

Remember that I mentioned local receptive fields? That is K , otherwise known as the *kernel*. This little gif is a visual representation of what the kernel does.

In short terms, it learns features. It 'slides' across the image and learns specific spatial patterns, like edges, curves or straight lines. In my network, I have 16 of these, each learning their own set of features.

2.2 Architecture Breakdown

2.2.1 Part 1: Convolutional Layers

Now that we are experts on kernels, we can talk a little about the architecture itself.

The input to my model are 28×28 grayscale images. As I mentioned, it then uses 16 filters to learn the features of the image.

Next up, I have a ReLU layer. This is a really simple operation that does a lot of heavy lifting. Essentially it keeps only positive neuron activations, which helps the network learn non-linear patterns. We can think of this as 'lighting up' only the pixels where a feature was detected.

Then we apply max pooling, which takes the max value in every 2×2 region, which basically just keeps the strongest signals. This reduces spatial size by half, and makes the network more robust to small shifts or distortions.

After the max pooling layer, I have ANOTHER convolution block, which takes those 16 feature maps and applies 32 new 3×3 filters. Now the network is combining earlier features into more complex shapes.

After that we have another ReLU and maxpool layer, so we're introducing non-linearity again and shrinking the image to 7×7 .

What's happened here in this part is that we've gone from raw pixels to abstract digit features.

2.2.2 Part 2: Fully Connected Layers

Now we've gone over from the learning part of the network, to the classification part. The goal here is to take the learned features and decide which digit the image represents.

The first part of this process involves converting the $32 \times 7 \times 7$ feature maps into one long vector of around 1,500 numbers.

Next up, we feed that long vector of numerical representation of features, into a linear layer which combines them into 64 high-level features. Think of this as: 'How do all these features relate to a specific digit?'

Again, we hit it with the ReLU, and then there is a dropout layer. Basically what this does is randomly turning off 25% of the neurons which in turn forces the network to not rely too much on any single feature which prevents overfitting.

Finally we have the last linear layer. This outputs an $(N, 10)$ values, which are called logits. Each logit corresponds to a digit class, and the class with the highest score is chosen as the prediction.

2.3 Briefly show the network

2.4 Training

If we take a step back from the low-level architecture and try to abstract it a little, I will talk about the actual goal of all machine learning tasks, which is to minimize some loss function.

2.4.1 Gradient Descent

We achieve this minimization by adjusting weights, which I briefly mentioned in the beginning. To put it short, weights are the parameters that control how much one neuron influences another one layer down. They determine which features the network deems important, and by continuously updating these weights, the network learns to make better predictions.

We adjust the weights by using gradient descent. Think of this as walking downhill in a foggy valley, the slope tells you which direction will reduce the loss fastest, and the step size is the learning rate, η .

So here, $\nabla \mathcal{L}$ is the gradient of the slope. A too high learning rate can actually make us overshoot the bottom of the valley, while a too low learning rate will just make it take forever to get there.

2.4.2 Epochs

There is also this thing called epochs. One epoch is just one full pass through the training dataset. During each epoch, the network sees every training example in smaller batches, and updates these weights gradually.

Repeating this process over multiple epochs allows the model to incrementally improve its predictions.

Just like anything, we can't just increase the number of epochs and call it a day. Too many epochs leads to overfitting, which means that the network has just memorized training data, and fails to predict accurately on unseen test data.

And vice versa, too few epochs leads to underfitting, which just means that the model doesn't learn anything and just is dumb.

2.5 Evaluation

But how can we know if a model is smart or good?

We don't want to use *just* accuracy, because if there happens to be 102 samples in the test set, and 101 of them are pictures of 1s, a dumb model can just guess 1 every time and still achieve an accuracy of close to 100%.

Therefore we combine *precision* and *recall* into one scoring metric, called the F1 score. Precision lets us know 'of all instances predicted positive, what percentage was actually positive?' and recall lets us know 'of all actual positive instances, what percentage did the model correctly identify?'. This score is what I use to evaluate my model.

2.6 Side note on overfitting: double descent

Earlier I mentioned that too much training can lead to overfitting, but that is kind of a lie. Belkin coined the term ‘double descent’ in 2019, and what he and his team discovered, was that more complex models actually can overcome the bias-variance tradeoff.

Normally, a developer would look at just this area (model size between 1 and 10), and say that this point in the middle is the optimal model size because we have the lowest error here. But if you just add complexity to the model, you can eventually overcome this hump, and get a much better model. This concept applies to CNNs, ResNets and transformers, just to mention a few.

But why does this happen? We don’t know.

3 Implementation

So on to implementation. First off we have all our imports here, and the dataset itself is just fetched from scikit-learn. We set a seed for reproducibility, and initialize a dict for our models. Just ignore this k for now, I’ll talk about it later, but remember that it is 5.

Next up we just split the dataset into X and y , which are our input features and target labels. Then we perform a 70-30 train-test split, which means that we will use 70% of our data for training and the rest for testing and final evaluation.

3.1 Visualization

What does the actual data look like? Here I have visualized all 10 digits so we can see what the model is actually working with.

It is important to know if the classes are balanced, which it looks like they are. If the dataset was imbalanced, it would lead to poor model performance. If that was the case, then I would employ SMOTE, or synthetic minority over-sampling technique, which basically just means using a model to create new instances within any given minority class.

4 K-fold Cross-Validation

I mentioned earlier that the learning rate and the amount of epochs impact the model performance. These are called hyperparameters. So how do I know which hyperparameters to use?

The answer is trial and error in a way. I do this systematically with a technique called K-fold Cross Validation.

We start with a set of hyperparameters, which I define as params here. GridsearchCV tries out all of these 15 combinations, five times each, totalling 75 fits.

To make sure our evaluation is robust, we run each combination five times. The training data is split into 5 equally sized ‘folds’. This way, our model is trained five times, each time using 4 folds for training and 1 fold for evaluation, which is why I didn’t split the data into a 70-15-15 train-eval-test split in the beginning. Every example is used for validation exactly once, and we get an average performance score that better reflects how the model will perform on unseen data.

This is usually a compute-heavy process, but since torch supports running computations on CUDA, and I have a 5090 I can ssh into, this process is roughly 10 to 20 times faster compared to my M3 chip.

5 Results

After the cross validation, we can see that the optimal initial learning rate is 0.0005 on 50 epochs, rendering an F1 score of 0.99, which is pretty good.

Running the model on unseen test data, we achieve an F1 score that is slightly better than the train/validation score.

Here I have a little confusion matrix showing predicted vs. true labels, and we can see that the model accurately predicts labels very nicely.

Finally, we can look at some of the misclassified examples, and honestly I don't blame the model for classifying wrong. Some of these digits are unrecognizable.